SUPPLEMENTARY MATERIAL

A PRELIMINARIES

In this section, we provide an overview of the fundamental concepts of large language models (LLMs) and continual learning (CL). We begin by introducing the notation used in this paper. Subsequently, we discuss the pre-training and downstream adaptation of LLMs, as well as mainstream LLM families (Appendix A.1), followed by an introduction to basic continual learning techniques studied by the community (Appendix A.2).

Notation. We denote scalars with lowercase letters, vectors with lowercase boldface letters, and matrices with uppercase boldface letters. The l_2-norm of vectors and the Frobenius norm of a matrix are represented by $\|\cdot\|_2$. For a vector $\boldsymbol{v} = [v_1, v_2, \cdots, v_n]^\top$, $\|\boldsymbol{v}\|_2 = (\sum_{i=1}^n v_i^2)^{1/2}$; for a matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$, $\|\boldsymbol{A}\|_2 = (\sum_{ij} A_{ij}^2)^{1/2}$. We use $\epsilon_{\mathcal{D}}$, $\mathcal{L}_{\mathcal{D}}$ to denote the error function, and loss function that is deployed for training, respectively, where the subscript is used to denote the error/loss measured by taking the expectation on the data distribution $\mathcal{D}$. We further use $\widehat{\mathcal{L}}_S$ to represent the empirical evaluation of the loss function $\mathcal{L}$ over the set of examples S. Probability and expectation are denoted by P and $\mathbb{E}$, respectively. We use $[m]$ to denote the set of positive integers up to m, $\{1, \cdots, m\}$.

A.1 Large Language Models

In the past two decades, neural language modeling has emerged as the dominant field of deep learning, marked by significant and rapid advancements. Primarily built on the transformer architecture, pre-trained language models (PLMs) like BERT have established a universal hidden embedding space through extensive pre-training on large-scale unlabeled text corpora. Following the pre-training and fine-tuning paradigms, PLMs exhibit promising performance across various natural language processing tasks after being fine-tuned upon small amounts of task-specific data [51, 133, 189]. Research on scaling laws indicates that increasing model size enhances the capacity of language modelss [84, 102]. By scaling parameters to billions or even hundreds of billions and training on massive text datasets, PLMs not only demonstrate superior language understanding and generation capabilities but also manifest emergent abilities such as in-context learning, instruction following, and multi-step reasoning, which are absent in small-scale language models like BERT [159, 250, 252, 253, 277]. These larger models are commonly referred to as Large Language Models (LLMs).

A.1.1 Pre-Training of LLMs. Pre-training is essential for language models to acquire broad language representations. Decoder-only models typically employ probability language modeling (LM) tasks during pre-training. LM, in this context, specifically refers to auto-regressive LM. Given a sequence of tokens $\boldsymbol{x} = [x_1, x_2, \cdots, x_N]$, LM predicts the next token x_t autoregressively based on all preceding tokens $\boldsymbol{x}_{<t} = [x_1, x_2, \cdots, x_{t-1}]$, and trains the entire network by minimizing the negative log-likelihood:

$$\mathcal{L}_{\mathrm{LM}}(\boldsymbol{x}) \triangleq -\sum_{t=1}^{N} \log P(x_t|\boldsymbol{x}_{<t}), \tag{2}$$

where $P(x_1|\boldsymbol{x}_{<1}) \triangleq P(x_1)$ is the unconditional probability estimation of the first token. The three most popular families of decoder-only models are GPT, PaLM, and LLaMA. The GPT family, developed by OpenAI, includes models such as GPT-2 [186], GPT-3 [22], ChatGPT [173], and GPT-4 [1]. Notably, GPT-3 was the first LLM to exhibit emergent abilities not found in smaller PLMs. Another notable family, Gemini, developed by Google, is comparable to the GPT family [194, 225]. While both GPT and Gemini families are closed-source, LLaMA, released by Meta, is currently the most popular open-source family of LLMs [230, 231]. The weights of these models are made available to the research community under non-commercial licenses.

Masked language modeling (MLM) task serves as a common pre-training objective for encoder-only models like BERT [51, 133]. In MLM, for the input sequence $\boldsymbol{x}$, a subset of input tokens $m(\boldsymbol{x})$ are masked and replaced with the special [MASK] token. The pre-training goal is to utilize the unmasked parts $\boldsymbol{x}_{\setminus m(\boldsymbol{x})}$ to predict the masked portions $m(\boldsymbol{x})$. In summary, the overarching goal of MLM is to minimize the negative log-likelihood:

$$\mathcal{L}_{\text{MLM}}(\boldsymbol{x}) \triangleq -\sum_{\widehat{x}\in m(\boldsymbol{x})} \log P(\widehat{x}|\boldsymbol{x}_{\setminus m(\boldsymbol{x})}). \tag{3}$$

Some encoder-decoder architecture models, such as T5 [189], also utilize Sequence-to-Sequence MLM task as the pre-training objective. They take masked sentences as encoder inputs and utilize the decoder to sequentially predict the masked tokens.

A.1.2 Adaptation of LLMs. LLMs are primarily trained to generate linguistically coherent text. However, this training may not align with human values, preferences, or practical needs. Furthermore, the pre-training data can be outdated, leading to knowledge cutoffs or inaccuracies. To address these issues, various computational paradigms such as Instruction Tuning (IT) [288], Model Refinement (MR) [47], and Model Alignment (MA) [174, 187] have been proposed. These approaches adapt LLMs to better meet diverse downstream tasks and user requirements.

Definition A.1 (**Instruction Tuning, IT**). *Let $h(\boldsymbol{x})$ be a language model that takes as input data $\boldsymbol{x}$, typically consisting of natural language instructions or queries. Instruction Tuning (IT) is a specialized training approach designed to enhance the model's ability to accurately and effectively respond to specific instructions. The objective of IT is to refine h by adjusting its parameters using a designated set of training examples $\mathcal{I} = \{(\boldsymbol{x}_i, \widehat{\boldsymbol{y}}_i)\}_{i=1}^N$ drawn from the IT data distribution $\mathcal{D}_{\mathcal{I}}$, where $\widehat{\boldsymbol{y}}_i$ represents the desired output for $\boldsymbol{x}$. This set is curated to target specific tasks or functionalities that require improved performance. Formally, IT seeks to find an optimal refined hypothesis h^* that satisfies:*

$$h^* \triangleq \arg\min_{h'} \mathbb{E}_{(\boldsymbol{x},\boldsymbol{y})\sim\mathcal{D}_{\mathcal{I}}} \left[-\log P(\widehat{\boldsymbol{y}}|\boldsymbol{x}, h')\right] \approx \arg\min_{h'} \sum_{i=1}^{N} -\log P(\widehat{\boldsymbol{y}}_i|\boldsymbol{x}_i, h'). \tag{4}$$

Remark. *The task of Model Alignment (MA) is usually formulated in the same problem definition as IT, with an alignment dataset of size M as $\mathcal{A} = \{(\boldsymbol{x}_a, \boldsymbol{y}_a, \widehat{\boldsymbol{y}}_a)\}_{a=1}^M$, where $\boldsymbol{y}_a$ represents the model's original decision for input $\boldsymbol{x}_a$, and $\widehat{\boldsymbol{y}}_a$ denotes the aligned decision that adheres to specified ethical guidelines or desired outcomes.*

Definition A.2 (**Model Refinement, MR**). *Suppose we have a model $h(\boldsymbol{x})$ taking data $\boldsymbol{x}$ (e.g., natural language queries) as inputs. Consider a size-N editing set $\mathcal{E} = \{(\boldsymbol{x}_e, \boldsymbol{y}_e, \widehat{\boldsymbol{y}}_e)\}_{e=1}^N$, where $\widehat{\boldsymbol{y}}_e$ denotes the true label of $\boldsymbol{x}_e$, but the model incorrectly outputs $\boldsymbol{y}_e$ for $\boldsymbol{x}_e$. Model Refinement (MR) aims to efficiently update the model from h to h' such that it correctly predicts the editing set $\mathcal{E}$, while preserving the original outputs outside $\mathcal{E}$. Formally, we aims to find h' satisfying*

$$h'(\boldsymbol{x}_0) = \begin{cases} \widehat{\boldsymbol{y}}_0 & \text{if } (\boldsymbol{x}_0, \widehat{\boldsymbol{y}}_0) \in \mathcal{E}, \\ h(\boldsymbol{x}_0) & o.w. \end{cases} \tag{5}$$

A.2 Continual Learning

Humans gradually accumulate knowledge and skills across tasks without significant performance decline on previous tasks [101, 153, 154, 175]. In contrast, machine learning models are usually data-centric, minimizing the training loss on the subsequent tasks will cause the model fail on the old ones, which phenomenon is phrased as *"catastrophic forgetting"*. Addressing this challenge is a focal point in continual learning research. The problem of efficiently adapting models to

a sequence of tasks without forgetting is extensively studied in the continual learning community [38, 178, 232, 237]. These studies are typically conducted under the following memory constraint of CL.

Definition A.3 (**Memory Constraint of Continual Learning**). *Suppose T sets of observations $\{S_t \sim \mathcal{T}_t\}_{t=1}^{T}$ come in as a sequence, where $\{\mathcal{T}_t\}_{t=1}^{T}$ denotes the T task distributions . At the learning stage $t > 1$, the sets of observations $\{S_i\}_{i=1}^{t-1}$ are not accessible (**strong**) or only partially accessible (**relaxed**).*

Remark. *In early stages of CL, works mostly focused on the strong memory constraint [4, 113, 124, 135]; as the research field progresses, more focus was put on relaxing the memory constraint to a small buffer for replay [24, 30, 193, 213]; some modern CL works completely discard the memory constraint but put focus on the computational budget [181, 233].*

A.2.1 Three Types of Continual Learning. There are three outstanding types of continual learning scenarios: task-incremental learning (TIL), domain-incremental learning (DIL), and class-incremental learning (CIL). To establish a groundwork for subsequent discussions (as illustrated in Table 3 and Section 6.2), we adhere to the conceptual framework proposed by [112, 232, 237] and offer formal definitions for these three continual learning scenarios.

Definition A.4 (**Task-Incremental Learning, TIL**). *Suppose T task distributions $\{\mathcal{T}_t\}_{t=1}^{T}$ come in as a sequence, where $\mathcal{T}_t$ denotes the joint distribution over the t-th task's input space and the label space $(\mathcal{X}_t, \mathcal{Y}_t)$. Denote $\mathcal{X} \triangleq \bigcup_{t=1}^{T} \mathcal{X}_t$ and $\mathcal{Y} \triangleq \bigcup_{t=1}^{T} \mathcal{Y}_t$ as the union of the input and label spaces, respectively. Under the memory constraint defined in Definition A.3, Task-Incremental Learning (TIL) aims to find the optimal hypothesis $h^* : \mathcal{X} \times [T] \to \mathcal{Y}$ that satisfies:*

$$h^* = \arg\min_{h} \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{T}_t} \left[\mathbb{1}_{h(\boldsymbol{x},t)\neq y} \right]. \tag{6}$$

Definition A.5 (**Domain-Incremental Learning, DIL**). *Suppose T domain distributions $\{\mathcal{D}_t\}_{t=1}^{T}$ come in as a sequence, where $\mathcal{D}_t$ denotes the t-th joint distribution over the shared input space and label space $(\mathcal{X}, \mathcal{Y})$. Under the memory constraint defined in Definition A.3, Domain-Incremental Learning (DIL) aims to find the optimal hypothesis $h^* : \mathcal{X} \to \mathcal{Y}$ that satisfies:*

$$h^* = \arg\min_{h} \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{D}_t} \left[\mathbb{1}_{h(\boldsymbol{x})\neq y} \right]. \tag{7}$$

Definition A.6 (**Class-Incremental Learning, CIL**). *Suppose T task distributions $\{\mathcal{T}_t\}_{t=1}^{T}$ come in as a sequence, where $\mathcal{T}_t$ denotes the joint distribution over the t-th task's input space and the label space $(\mathcal{X}_t, \mathcal{Y}_t)$. Denote $\mathcal{X} \triangleq \bigcup_{t=1}^{T} \mathcal{X}_t$ and $\mathcal{Y} \triangleq \bigcup_{t=1}^{T} \mathcal{Y}_t$ as the union of the input and label spaces, respectively. Under the memory constraint defined in Definition A.3, Class-Incremental Learning (CIL) aims to find the optimal hypothesis $h^* : \mathcal{X} \to [T] \times \mathcal{Y}$ that satisfies:*

$$h^* = \arg\min_{h} \sum_{t=1}^{T} \mathbb{E}_{(\boldsymbol{x},y)\sim\mathcal{T}_t} \left[\mathbb{1}_{h(\boldsymbol{x})\neq(t,y)} \right]. \tag{8}$$

Remark. *In TIL, it is common to have a shared input space $\mathcal{X} = \mathcal{X}_t, \forall t \in [T]$, but the space of the label distribution $\mathcal{Y}_t$ can be distinct ($\mathcal{Y}_i \cap \mathcal{Y}_j = \emptyset, \forall i \neq j$), partially shared ($\mathcal{Y}_i \cap \mathcal{Y}_j \neq \emptyset, \exists i \neq j$), or shared across different tasks ($\mathcal{Y} = \mathcal{Y}_t, \forall t \in [T]$). In DIL, the tasks are defined in the same format, i.e., same input space $\mathcal{X}$ and same output space $\mathcal{Y}$. During the inference, no task IDs are provided for the hypothesis, which means the continual learning model needs to capture the pattern between the domain-invariant features and the labels. DIL is commonly perceived as more difficult than TIL. CIL is commonly viewed as the most challenging continual learning scenario, as the model needs to infer the label and the task ID at the same time. Another possible formulation of CIL is to represent it as DIL but the output label spaces are disjoint, $\mathcal{Y}_i \cap \mathcal{Y}_j = \emptyset, \forall i \neq j$.*

A.2.2 Techniques of Continual Learning. The objective of CL is to find a hypothesis that minimizes risk across all tasks/domains. Consider DIL as an example [213], at t-th learning stage, the ideal training objective $\mathcal{L}(h)$ is defined as

$$\mathcal{L}(h) \triangleq \underbrace{\sum_{i=1}^{t-1} \mathcal{L}_{\mathcal{D}_i}(h)}_{\text{past domains}} + \underbrace{\mathcal{L}_{\mathcal{D}_t}(h)}_{\text{current domain}} . \tag{9}$$

The objectives for past domains are often challenging to measure or optimize due to the memory constraints (Definition A.3). Therefore, the core of designing CL algorithms lies in identifying a proxy learning objective for the first term without violating the memory constraint. Existing CL techniques can be roughly categorized into 5 groups: (i) replay-based, (ii) regularization-based, (iii) architecture-based, (iv) optimization-based, and (v) representation-based [237]. Here, we provide a concise yet comprehensive introduction to the first three categories of continual learning techniques, as they find extensive application in continually learning large language models.

Replay-Based Methods. Replay-based methods adopt the relaxed memory constraint by keeping a small buffer of observed data $\{M_i\}_{i=1}^{t-1}$ for each task $\mathcal{T}_i$. Formally, they seek to optimize the following empirical training objective:

$$\widehat{\mathcal{L}}_{\text{replay}}(h) \triangleq \underbrace{\sum_{i=1}^{t-1} \widehat{\mathcal{L}}_{M_i}(h)}_{\text{proxy for past domains}} + \underbrace{\widehat{\mathcal{L}}_{S_t}(h)}_{\text{current domain}} , \tag{10}$$

where $\widehat{\mathcal{L}}_S$ denotes the empirical loss term evaluated on the set of examples S. Often regarded as a simplistic solution to CL, replay-based methods may theoretically lead to loose generalization bounds [213]. Despite this, they are valued for their simplicity, stability, and high performance, even with a small episodic memory [30, 195]. For instance, DER++ [24] demonstrates consistent performance enhancement by replaying a small set of past examples along with their logits (known as dark experience replay). ESM-ER [204] introduces error sensitivity modulation (ESM) to mitigate abrupt representational drift caused by high-error new examples. A significant focus in replay-based CL is enhancing sample efficiency for buffer maintenance. For instance, [193] prioritizes exemplar selection based on herding to accurately model class mean throughout class-incremental learning. [295] propose storing low-fidelity examples to achieve memory-efficient exemplar set maintenance.

Regularization-Based Methods. Suppose $h_{\boldsymbol{\theta}_{t-1}}$ is the hypothesis yielded after the $t-1$-th stage of training, parameterized by $\boldsymbol{\theta}_{t-1}$. Regularization-based methods utilize a regularization term as a proxy for past domain losses, determined by the distance in the parameter space.

$$\widehat{\mathcal{L}}_{\text{reg}}(h_{\boldsymbol{\theta}}) \triangleq \underbrace{\lambda \cdot \|\boldsymbol{\theta} - \boldsymbol{\theta}_{t-1}\|_{\Sigma}}_{\text{proxy for past domains}} + \underbrace{\widehat{\mathcal{L}}_{S_t}(h_{\boldsymbol{\theta}})}_{\text{current domain}} , \tag{11}$$

where $\|\boldsymbol{v}\|_{\Sigma} = \boldsymbol{v}^{\top}\Sigma\boldsymbol{v}$ is the vector norm evaluated on a positive-semi-definite matrix Σ, and λ is the regularization coefficient, a hyper-parameter introduced to balance the past knowledge retention and current knowledge learning. The matrix Σ introduced is to measure the different level of importance of each parameters and their correlations in retaining the past knowledge. In practice, to reduce computational overhead, diagonal matrices are often designed to encode only the importance of each parameter. For example, Elastic Weight Consolidation (EWC) [113] adopts a Bayesian perspective, using diagonal values from the Fisher Information Matrix (FIM) as an approximation for the Hessian matrix of parameters. This forms a sequential Maximize A Posteriori (MAP) optimization for continual learning.

Memory Aware Synapses (MAS) [4] computes parameter importance in an online and unsupervised manner, defining importance by accumulated absolute gradient during training. It is also worth noting that when $\Sigma = \boldsymbol{I}$ degenerates to an identity matrix, the regularization term simplifies to a basic l_2-penalty term, equally penalizing each parameter, which can be surprisingly effective in some cases of continual LLMs [197].

Architecture-Based Methods. Expanding the network architecture dynamically to assimilate new knowledge is deemed the most efficient form of CL [248, 249]. This method primarily tackles adaptation challenges and can achieve zero-forgetting when task IDs are available during inference or can be correctly inferred [71, 256]. However, due to the difficulty of task ID inference, architecture expansion is predominantly utilized in TIL but is scarcely explored in DIL or CIL. Progressive Neural Networks (PNN) [201] proposes learning laterally connected neurons as new tasks arise, ensuring non-forgetting and enabling transfer of previously learned neurons for future tasks. In conjunction with pre-trained backbone large models like ViT [54], CoLoR [256] trains various low-rank adaptation (LoRA) [86] modules for different tasks. It estimates and stores prototypes for each task and utilizes the natural clustering ability of the pre-trained model during testing to infer task IDs, selecting the corresponding LoRA component for prediction generation. In the domain of continual LLMs, architecture expansion has resurged in popularity following the rise of parameter-efficient fine-tuning (PEFT) [50, 86, 211], a topic we will delve into shortly [96, 100, 118, 177, 240, 257, 272, 273].

B EVALUATION PROTOCOLS AND DATASETS

In Appendix B.1, we review common continual learning evaluation metrics and provide formal definitions. In Appendix B.2, we introduce metrics designed specifically for continual LLMs. Finally, in Appendix B.3, we outline the datasets available for each discussed topic.

B.1 Evaluation Metrics of Continual Learning

In the realm of conventional continual learning, where task streams take the form of classification, many metrics rely on the concept of Accuracy Matrix [136, 213]. Extending this notion to the context of continually learning LLMs, we introduce the **Performance Matrix** $\boldsymbol{P} \in \mathbb{R}^{T\times T}$, where T represents the total number of training stages. Each entry of $\boldsymbol{P}$ corresponds to a performance metric evaluated on the models, such as perplexity on pre-training data [35, 69, 100], zero-shot/few-shot evaluation metrics on downstream data without fine-tuning [9, 42, 48, 172, 199, 258], fine-tuned accuracies on downstream tasks [5, 35, 96, 183], and probing accuracies from fine-tuning add-on components evaluated on downstream tasks [144, 223, 299]. In $\boldsymbol{P}$, $P_{i,j}$ denotes the model's performance after training on task i and evaluating on task j. With this Performance Matrix definition, we introduce the primary evaluation protocols widely adopted.

Overall Performance (OP). The Overall Performance (OP) [106, 286, 291] is a natural extension of the concept of Average Accuracy [136, 213]. The OP measured up until training stage t is the average performance of the model trained right after the stage t. Denote it as OP_t and we have:

$$\mathrm{OP}_t \triangleq \frac{1}{t}\sum_{i=1}^{t} P_{t,i}. \tag{12}$$

As noted in [213], the OP corresponds to the primary optimization objective defined in Definition A.4, A.5, and A.6. In much of the continual learning literature, once all T tasks are completed, the final OP (OP_T) is reported, with the subscript $_T$ often omitted for brevity. In some works, OP is weighted by the importance of tasks $\widetilde{\mathrm{OP}} \triangleq \frac{1}{T}\sum_{i=1}^{T} w_i P_{t,i}$, where $w_i = N_i / \sum_{j=1}^{T} N_j$ represents the ratio of data. In some literature, $\widetilde{\mathrm{OP}}$ is referred to as "example accuracy" [37], "whole accuracy" [217], or "edit success rate" in CMR [74].

Forgetting (F). Define F_t as the forgetting up to task t, which represents the largest performance drop observed throughout the training process, averaged over t training stages:

$$F_t \triangleq \frac{1}{t-1} \sum_{j=1}^{t-1} \left[\max_{l \in [t-1]} \{P_{l,j} - P_{t,j}\} \right]. \tag{13}$$

Typically, researchers report the average forgetting $F = F_T$ at the end of the entire training process. Forgetting quantifies the impact of learning new tasks on previously acquired knowledge. Ideally, a robust continual learning framework should achieve **Backward Transfer (BWT)**, where learning new tasks enhances performance on prior tasks. This enhancement is typically measured by negating the forgetting, thus indicating an improvement in performance on earlier tasks. The concepts of Forgetting and Backward Transfer underpin various evaluation metrics, such as knowledge retention [100], performance on unchanged knowledge [95], average increased perplexity (AP^+) [184], and test and edit retention rate in CMR [74].

Forward Transfer (FWT). Forward Transfer measures the generalization ability of the continual learning algorithms. Formally, forward transfer FWT_t up to training stage t is defined as

$$\text{FWT}_t \triangleq \frac{1}{t-1} \sum_{i=2}^{t} P_{i-1,i} - b_i, \tag{14}$$

where b_i is the baseline performance of the model evaluated on task i before undergoing continual learning. Strictly speaking, the definition of b_i is not the same as defined in the previous work [136, 213], where it is used to denote the performance of a random initialization of the model. Additionally, we extend the notation of forward transfer in the vertical direction to represent the performance improvement on downstream tasks resulting from domain-adaptive pre-training (see Table 2). Forward Transfer is alternatively referred to as temporal generalization [100] or knowledge transfer [116] in some literature. In this section, we introduce the evaluation protocols and datasets for continul LLMs.

B.2 Continual LLMs' Evaluation Protocols

LAnguage Model Analysis (LAMA). LAnguage Model Analysis (LAMA) is an evaluation framework designed to *probe the world knowledge* embedded in language models [179]. It converts each world fact into a cloze statement, which is then inputted into the language models to predict the correct answer. LAMA has been extended for continual pre-training, particularly for those under the temporal shifts [95, 96]. In CKL, three LAMA benchmarks are constructed for different dimensions: InvariantLAMA assesses knowledge retention on time-invariant facts, UpdatedLAMA focuses on knowledge update, and NewLAMA evaluates knowledge acquisition [96].

Forgotten / (Updated + Acquired) Ratio (FUAR). As the performance of a pre-trained LLM is decomposed into a fine-grained set in CKL [96], OP becomes a too general metric and cannot accurately reflect the balance and trade-offs of the model's behavior. To address this issue, CKL proposes a joint evaluation metric FUAR (Forgotten / (Updated + Acquired) Ratio) for continual pre-training. A FUAR value of 1 represents an equal trade-off between the knowledge forgetting and knowledge learning: for each piece of updated or acquired knowledge, one piece of time-invariant knowledge is forgotten on average. A FUAR less than 1 suggests high learning efficacy, where more than one piece of knowledge is acquired at the expense of forgetting one piece of time-invariant knowledge.

X-Delta. In TRACE [241], the authors propose a set of "X-Delta" metrics for continual instruction tuning, quantifying the forward transfer on specific abilities of LLMs. Let's denote a set of M datasets $\{X_1, X_2, \cdots, X_M\}$ for task X. The baseline performances of the pre-trained LLM evaluated on these tasks are denoted as $\{b_1^X, \cdots, b_M^X\}$. The model

undergoes continuous fine-tuning on a different set of tasks, distinct from those used for evaluation. Throughout the sequential training process, the performance of the model after learning task t on evaluation tasks X_i is $R^X_{t,i}$. The X-Delta ΔR^X_t after learning task t is defined as:

$$\Delta R^X_t \triangleq \frac{1}{M} \sum_{m=1}^{M} (R^X_{t,i} - b^X_i). \tag{15}$$

In the public TRACE benchmark, the authors construct three sets of evaluation tasks to benchmark the ability of LLMs, including *general ability*, *instruction following*, and *safety* [241].

NLG Score. In continual model alignment, three prominent metrics used to evaluate different aspects of Natural language generation (NLG) are BLEU-4 [176], METEOR [12], and ROUGE-L [126]. BLEU-4 [176], designed for machine translation (MT), evaluates the precision of n-grams between the machine-generated and reference texts, focusing especially on four-word sequences to gauge fluency and adequacy. METEOR [12] also targets MT but aims to improve correlation with human judgment by considering synonyms and stemming, thus providing a more nuanced assessment of translation quality. On the other hand, ROUGE-L [126] is commonly applied in summarization tasks, assessing the longest common subsequence between the generated summary and a set of reference summaries, effectively measuring the recall of essential content. Each metric has its strengths and is tailored to specific kinds of language processing tasks, reflecting different dimensions of text generation quality.

B.3 Datasets

In this section, we provide a comprehensive review of the datasets available for benchmarking continual LLMs, as illustrated in Table 4. We intentionally exclude datasets used for domain-adaptive pre-training LLMs in vertical domains such as legal, medical, and financial, unless they are specifically designed for continual domain-adaptive pre-training. Furthermore, we omit datasets used in general continual fine-tuning, as they have already been extensively studied in existing works [17, 105].

Datasets for Continual Pre-Training (CPT) and Domain Adaptive Pre-Training (DAP). Current research lacks a widely recognized benchmark for evaluating continual pre-training LLMs under temporal shifts. TimeLMs utilizes a series of Twitter corpora collected until 2022, sequentially pre-training RoBERTa models quarterly [137]. CC-RecentNews, adopted as unlabeled pre-training data for LMs in CKL [96], consists of recent news and serves as a single-stage dataset. Additionally, CKL introduces InvariantLAMA, NewLAMA, and UpdatedLAMA to assess the principles of continual knowledge learning. TWiki, a dataset derived from the articles of Wikipedia between August and December 2021, is curated and cleaned in TemporalWiki [95]. This dataset facilitates the exploration of incremental learning by providing the Diffsets between neighboring snapshots. For works that study the content-level distributional shifts in CPT and DAP, researchers often resort to a similar set of publicly available datasets [134, 169, 270] to construct their own test beds for continual learning algorithms. The *DAPT dataset, developed by [72], comprises four domains: BioMed and Computer Science from S2ORC [134], News from [283], and Reviews from [78]. In *DAPT's original study, each domain undergoes individual domain adaptive pre-training stages to demonstrate the universality of DAP's effectiveness. Subsequent works, such as ELLE [184] and Recyclable Tuning [183], follow suit by employing these domains for multi-stage CPT. DEMix [71] presents another large-scale dataset, featuring eight semantic domains with over 73.8 billion tokens. Alongside the training set, it includes eight additional datasets for validating the generalization ability of LLMs. On a smaller scale, *CPT [104] and *DAS [107] datasets consist of four and eight domains, respectively,

with approximately 3.12 million examples and a size of 4.16GB each. These datasets are constructed similarly to the aforementioned ones.

Datasets for Continual Instruction Tuning. Measuring the effectiveness of CIT is crucial, particularly because traditional evaluation metrics may not be suitable for LLMs: many of them are overly simplistic and fail to comprehensively assess the model's ability to learn continually. New benchmarks and metrics are required to evaluate both the retention of old knowledge and the integration of new instructions. TRACE [241] stands as a continual learning benchmark designed specifically for LLMs, encompassing diverse tasks such as multilingual capabilities, code generation, and mathematical reasoning. CITB [292] represents another benchmark for CIT, incorporating both learning and evaluation protocols. It in addition demonstrates that replay generally yields the best performance across all methods. CoIN [32] extends the benchmark to MLLMs, incorporating a balanced and diverse set of instructions from vision-language datasets.

Datasets for Continual Model Refinement. Most datasets for continual model refinement can be categorized into two types [152]: fact checking and question answering. For fact checking, models are asked to verify the truthfulness of certain claims, typically modeled as a classification task. Key datasets include FEVER [228] (used by [47, 76]) and VitaminC [206] (used by [164]), both sourced from Wikipedia. For question answering, models are tasked with providing specific answers instead of choices. Zero-shot Relation Extraction (zsRE) [117] is the most widely employed dataset for this purpose [45, 74–76, 157, 158], alongside Natural Questions (NQ) [114] and T-rex [59]. [157] adapted zsRE with additional counterfactuals to create the more challenging CounterFact dataset, used by [45, 85, 280]. Beyond these two categories, SCOTUS [28] is also utilized [74] in the assessment of continual model refinement through a document classification task for U.S. Supreme Court cases into 11 topics.

Datasets for Continual Model Alignment. In the domain of reinforcement learning with human feedback (RLHF), several datasets are commonly employed across different studies to evaluate the adaptation and effectiveness of models under varying scenarios and continuous learning conditions. The IMDB [149] and HH-RLHF [11] dataset, as introduced in [286] within their study on continual learning through optimal policy fitting, leverages data gathered from interactive RL scenarios to model human preferences dynamically. Similarly, the Reddit TL;DR dataset [234] used by [286, 287] is focused on text summarization, providing a robust platform for testing the longevity and adaptability of learning algorithms under evolving conditions. Lastly, Common Sense QA [18, 41, 115], Reading Comprehension [56, 190], and Translation [19], which are utilized in [128] are selected to assess the challenges of aligning RL agents with human expectations without incurring significant performance penalties. Each of these datasets is pivotal in advancing the understanding of continual learning and the interplay between human feedback and machine learning adaptation.

Datasets for Continual Multimodal Large Language Models. Following LLaVA [131], many MLLMs adopt the pattern of instruction tuning to enable assessing alignment with human intention and knowledge preservation for reasoning. Thus, traditional tasks like image classification can be transformed to VQA tasks to evaluate the ability of MLLMs, which are otherwise challenging to assess using conventional methods. Several benchmarks have been proposed to evaluate the CL method for MLLMs. MCIT [77] proposes the first continual instruction tuning benchmarks, Benchmark1 and Benchmark2. The difference between benchmark1 and benchmark2 is that benchmark2 includes Multi-task Joint Instruction Tuning, which aims to explore whether multi-task joint instruction tuning improves the model's continual learning ability. [284] proposes EMT, the first classification evaluation framework to investigate catastrophic forgetting in MLLMs. [32] presents a comprehensive benchmark CoIN, spanning 8 task categories and evaluating MLLMs from two perspectives: Instruction Following and General Knowledge, which assess the alignment

with human intention and knowledge preserved for reasoning, respectively. [296] constructs two datasets, UPMC-Food101-CMML and MM-IMDb-CMML to benchmark the novel CMML task, which means the data of certain modalities is missing during continual fine-tuning. UPMC-Food101-CMM contains 101 food categories and 61,142 training, 6,846 validation, and 22,716 test image-text pairs. MM-IMDb-CMML is a multi-label classification dataset across 27 distinct movie genres, consisting of 15,552 training, 2,608 validation and 7,799 test image-text pairs.